# CS623 Assignment 2: CUDA Programming

Dweep Joshipura (Roll No. 230395)

## 1 Problem 1: 3D convolution

**Implementation.** The basic kernel uses one thread per output element and reads the input and filter directly from global memory. The optimized kernel uses 8×8×8 thread blocks that load a 12×12×12 input tile (with halo) into shared memory, keeps the 5×5×5 filter in constant memory, and fully unrolls the filter loops. Out-of-range elements are treated as zero, and both kernels are checked against the single-threaded CPU version.

**Results.** Table 1 gives means over 10 runs for $N \in \{64, 128, 256\}$. GPU and CPU outputs matched in all runs.

| $N$ | CPU | Basic (ms) | | Optimized (ms) | | Speedup | |
|---|---|---|---|---|---|---|---|
| | (ms) | kernel | end-to-end | kernel | end-to-end | kernel | end-to-end |
| 64 | 65.4 | 0.517 | 1.97 | 0.052 | 1.46 | 10.0× | 1.35× |
| 128 | 496.1 | 2.111 | 10.75 | 0.323 | 8.75 | 6.5× | 1.23× |
| 256 | 3849.2 | 15.346 | 80.82 | 2.505 | 67.25 | 6.1× | 1.20× |

**Table 1:** Problem 1: mean times over 10 runs. End-to-end includes host–device copies.

The optimized kernel is 6–10× faster than the basic one, since each input element loaded into shared memory is reused by up to 125 outputs and the filter reads become constant-memory broadcasts. End-to-end the gain is only 1.2–1.35×, because the PCIe transfers dominate once the kernel is fast (at $N = 256$ the optimized kernel is 2.5 ms of a 67 ms total). Compared with the CPU, the optimized GPU path is about 45× ($N = 64$) to 57× ($N = 256$) faster end-to-end.

## 2 Problem 2: 10-D grid search

The search space is $13^{10} \approx 1.4 \times 10^{11}$ points (13 values per dimension), of which 11 608 satisfy all ten constraints.

**Implementation.**

- **v0:** the given serial C code.
- **v1 (vanilla port):** one thread per grid point (index decoded in mixed radix), problem data in constant memory, hits appended with `atomicAdd` to a bounded buffer; the space is processed in chunks of $2^{28}$ points over several launches, and the host writes the file.
- **v2 (optimized):** one thread per $(x_1..x_7)$ prefix, which reuses the partial sums while looping over $x_8..x_{10}$; hits are reserved with one warp-aggregated `atomicAdd`. *Pruning:* all coefficients and grid values are non-negative, so partial sums only grow and a loop stops once a constraint is already violated (disabled automatically if the inputs are not non-negative).
- **v3 (Thrust):** `thrust::copy_if` over a `counting_iterator` with a predicate functor, in chunks of $2^{25}$ indices; hit indices are decoded on the host.

**Results.** Table 2 compares the versions. Timing covers the search, copies back and file writing (not start-up). Every version finds 11 608 points, and its sorted output is identical to v0's.

| Version | Description | Runs | Median (s) | Min–Max (s) | vs. v0 | vs. v1 |
|---|---|---|---|---|---|---|
| v0 | Serial CPU (reference) | 1 | 657.388 | – | 1.0× | 0.05× |
| v1 | Vanilla CUDA port | 5 | 29.928 | 29.927–30.155 | 22.0× | 1.0× |
| v2 | v2, no pruning | 5 | 5.542 | 5.542–5.543 | 118.6× | 5.4× |
| v2 | v2 + pruning (final) | 3 | 0.140 | 0.139–0.140 | 4701.7× | 214.0× |
| v3 | Thrust | 5 | 40.235 | 40.112–64.975 | 16.3× | 0.74× |

**Table 2:** Problem 2: total time on the A40 (v0 on the server CPU). v2 + pruning is the final version (block size 128, 3 runs).

v2 is 5.4× faster than v1 even without pruning (partial-sum reuse, far fewer atomics), and pruning gives a further 40× (5.54 s to 0.14 s) because most of the space is cut off early in the inner loops. The Thrust version is slower than v1 (0.74×): it re-evaluates every point from scratch and cannot prune, and `copy_if` adds extra passes. The first v3 run (65 s) was an outlier; the others were about 40 s.

**Launch configuration and pruning ablation** (Figure 1). v1 is insensitive to the block size (about 30 s for 64–1024 threads; the 39 s median at 128 reflects a noisy run). v2 is also flat from 64 to 512 threads. With 1024 threads v2 fails to launch: it uses 100 registers per thread, and 1024 × 100 exceeds the 65 536 registers available to one block. Pruning gives a 36–41× gain at every block size.

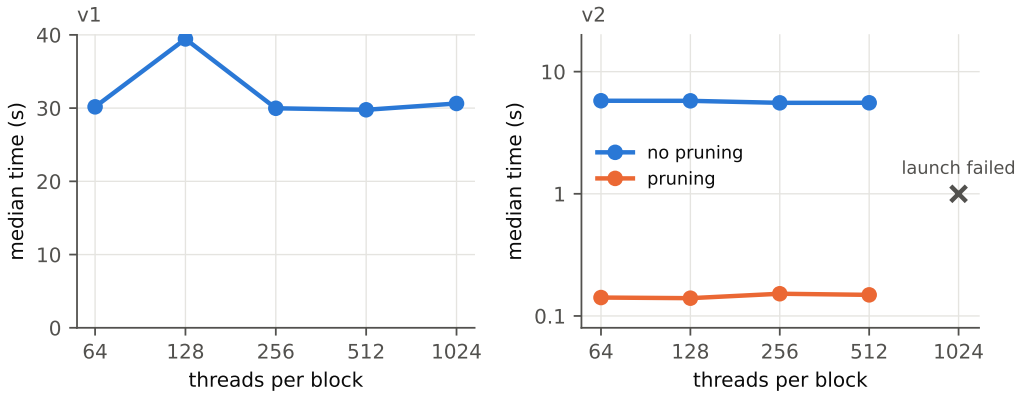



**Figure 1:** Median time versus threads per block (3 runs per point).

## 3 Problem 3: Counting Bloom filter

**Implementation.** The filter is a raw array of 8-bit counters, packed four per 32-bit word and updated with an `atomicCAS` loop on the word (CUDA has no 8-bit CAS). There is one thread per key (grid-stride loop, 256 threads per block), with separate `insert`, `delete` and `search` kernels and $k = 3$ hash functions (`hash1`–`hash3`). Duplicates in the insert and delete traces are removed on the host. A delete only decrements if all three counters are non-zero, so deleting an absent key does nothing. The *blocked* variant adds `hash4`, which picks a block of 32 counters (32 bytes, one L2 sector); `hash1`–`hash3` choose counters inside that block. `check_result()` replays the operations on a `std::unordered_set` and counts false negatives (none occurred in any run) and false positives.

**Trace and metrics.** The traces have $10^6$ insert keys (950 000 distinct), $10^6$ search keys (50%

inserted), and 300 000 delete keys (285 000 distinct). Unless stated otherwise $N = 20 \times 10^6$ counters. The false-positive rate is false positives divided by all searches; because half of the searches are for absent keys, it cannot exceed 50%.

### 3.1 Performance and false positives

| Kernel | Vanilla (ms) | Blocked (ms) | Blocked / vanilla |
|---|---|---|---|
| Insert (950 000 keys) | 1.125 ± 0.009 | 1.097 ± 0.034 | 0.98 |
| Search after insert ($10^6$) | 0.231 ± 0.003 | 0.173 ± 0.002 | 0.75 |
| Delete (285 000 keys) | 0.379 ± 0.002 | 0.347 ± 0.013 | 0.92 |
| Search after delete ($10^6$) | 0.209 ± 0.001 | 0.161 ± 0.003 | 0.77 |

**Table 3:** Kernel times (mean ± s.d. of 5 runs), $N = 20$M.

Blocked search is 25% faster and delete 8% faster, while insert is unchanged (Table 3); insert time is dominated by the `atomicCAS` loop.

| | After insert (%) | | After delete (%) | |
|---|---|---|---|---|
| $N$ | Vanilla | Blocked | Vanilla | Blocked |
| 1M | 41.806 | 40.752 | 41.979 | 40.386 |
| 5M | 4.103 | 4.398 | 2.319 | 2.708 |
| 10M | 0.773 | 1.024 | 0.396 | 0.613 |
| 20M | 0.119 | 0.229 | 0.055 | 0.144 |

**Table 4:** False-positive rate (% of all searches) versus filter size $N$.

The price is a higher false-positive rate (Table 4, Figure 2): at $N = 20$M, blocked gives 0.229% against 0.119% after the inserts. At $N = 1$M ($\approx$ one counter per key) both filters are saturated at about 41%, and the rate falls steeply as $N$ grows.

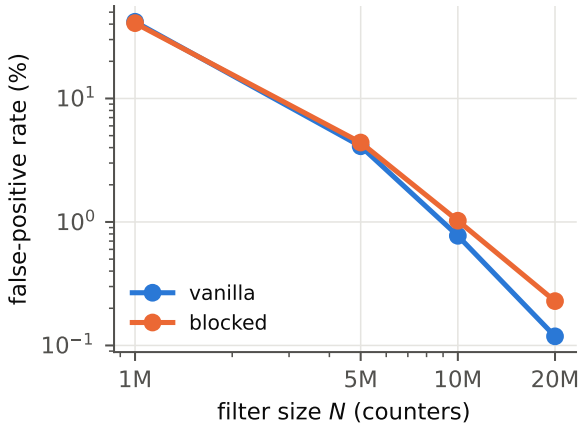



**Figure 2:** False-positive rate after the inserts versus $N$.

### 3.2 Block size ablation

Larger blocks lower the false-positive rate (0.315% to 0.134%, approaching vanilla's 0.119%) but make search slower (0.170 ms to 0.223 ms, approaching vanilla's 0.231 ms). With 32 counters (32 B)

| Block size | Bytes | FP after insert (%) | Search (ms) | Insert (ms) |
|---|---|---|---|---|
| 16 counters | 16 | 0.315 | 0.170 | 1.074 |
| 32 counters | 32 | 0.229 | 0.175 | 1.040 |
| 64 counters | 64 | 0.183 | 0.195 | 1.070 |
| 128 counters | 128 | 0.153 | 0.208 | 1.065 |
| 256 counters | 256 | 0.134 | 0.223 | 1.080 |
| vanilla (no blocks) | – | 0.119 | 0.231 | 1.125 |

**Table 5:** Blocked variant, $N = 20$M, 3 runs per setting; last row is the vanilla filter.

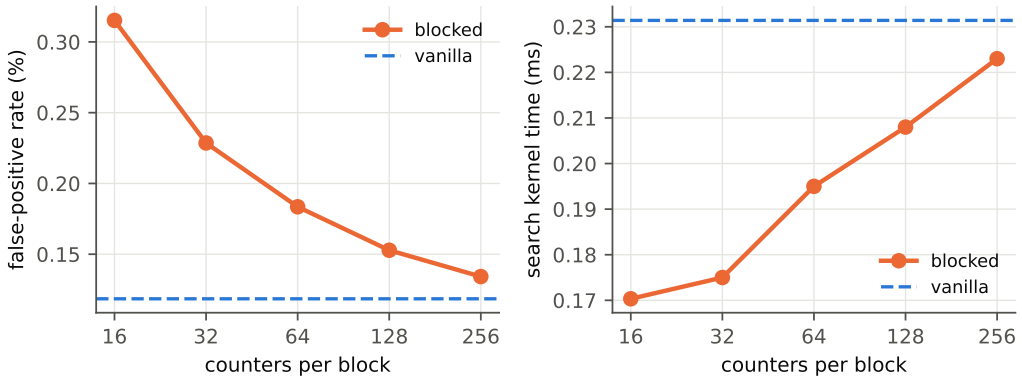



**Figure 3:** Effect of the block size: false positives (left) and search time (right).

all three counters of a key lie in a single sector. This is consistent with 32 counters being a good trade-off, and it is the default.

### 3.3 Nsight Compute profile

The first `insert` and first `search` kernel launches were profiled for both variants at $N = 20$M (`ncu --set full`); delete was not profiled.

| | Vanilla | | Blocked | |
|---|---|---|---|---|
| Metric | insert | search | insert | search |
| Duration | 1040 $\mu$s | 219 $\mu$s | 988 $\mu$s | 160 $\mu$s |
| DRAM throughput (% of peak) | 64.8 | 79.7 | 62.0 | 74.6 |
| Achieved DRAM bandwidth (GB/s) | 451 | 554 | 431 | 519 |
| Arithmetic intensity (int. ops/byte) | 0.18 | 0.58 | 0.19 | 0.81 |
| Achieved occupancy (%) | 85.5 | 91.6 | 83.5 | 81.7 |
| Warp efficiency (%) | 88.7 | 88.9 | 87.2 | 87.7 |
| L2 hit rate (%) | 17.4 | 11.9 | 27.6 | 29.6 |
| Long-scoreboard stall (cycles/instr.) | 1311 | 357 | 1342 | 254 |

**Table 6:** Nsight Compute metrics. Arithmetic intensity is thread-level integer instructions per DRAM byte (the kernels execute no floating-point instructions). Warp efficiency is the average number of active threads per warp divided by 32.

All four kernels are memory bound: the profiler reports memory throughput (62–80% of peak DRAM bandwidth) far above compute throughput (1–4%). The blocked variant has the better L2 hit rate (28–30% against 12–17%) and moves less data from DRAM (search 83 MB against 122 MB, insert 426 MB against 469 MB), which is consistent with its faster search.

**Top three bottlenecks reported by the profiler:**

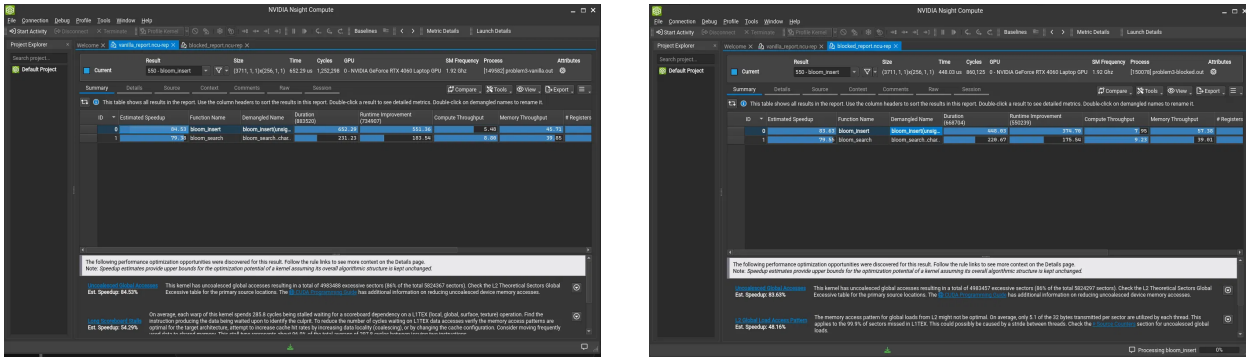
1. **Uncoalesced global accesses.** Counter lookups are random, so 85%/80% (vanilla insert/search) and 85%/80% (blocked) of the transferred sectors are excess; estimated speedup if fixed: 85.12%/77.97% (vanilla), 84.98%/77.24% (blocked).

2. **Long-scoreboard stalls.** Warps wait on L1TEX/global loads: 1311.2/356.9 cycles per issued instruction (vanilla insert/search) and 1341.8/254.4 (blocked).

3. **Poor DRAM/L2 access pattern.** Only 5.1–5.6 of every 32 bytes in a fetched sector are used, and 90.4%/89.6% of sectors miss L2 in vanilla versus 71.3%/72.1% in blocked.

Atomics were not flagged separately. The generic "compute pipelines under-utilized" rule is also reported, but it only restates that the kernels are memory-latency bound.

I also opened it in Nsight Compute GUI on RTX 4060 Laptop GPU, with same conclusions.



**(a)** Vanilla **(b)** Blocked

**Figure 4:** Nsight Compute GUI for Bloom filter kernels @ N = 20M

# 4 Setup

Experimentation was run locally on my laptop's NVIDIA RTX 4060 with an Intel Core i7-14650HX.

Final results were run on the GPU3 server: NVIDIA A40 (sm_86, 46 GB), Intel Xeon Silver 4208 (2.1 GHz base), CUDA 12.6, driver 590.48.

All code is built with the provided `Makefile` (`make`; `ARCH=sm_86` by default, override with `make ARCH=sm_80`). Problem 2 uses `--fmad=false`, and v3 additionally uses `-Xcompiler -ffp-contract=off`, so that results match the CPU code bit for bit.

Problem 2 times are wall-clock; Problem 1 and 3 kernel times use CUDA events. Run counts are given with each table; Problem 2 reports medians because the server is shared and a few runs were up to 2× slower.

# 5 Acknowledgement

AI was used for parts of the code, debugging, refactoring, and the report.